Jogo no terminal

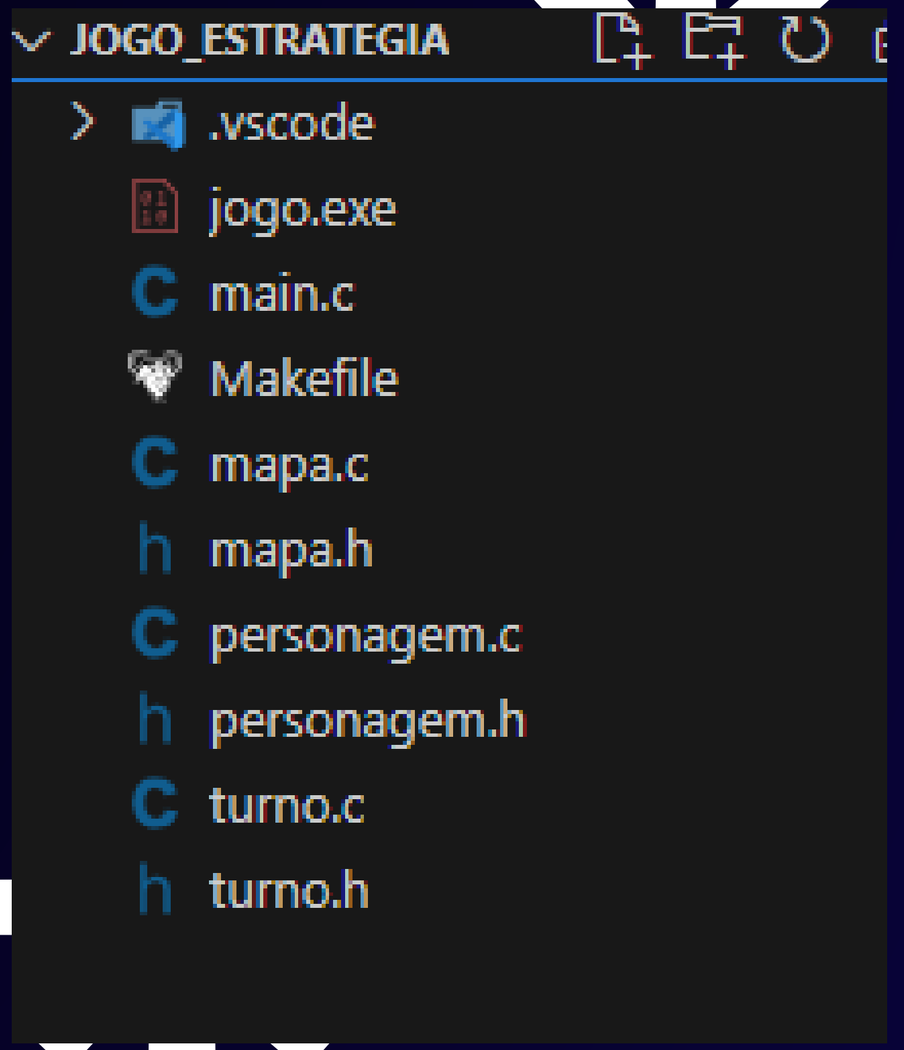
Aluno: Arthur Bertoli Silva
Estrutura de dados II
09/07/2025

Visão Geral

Um jogo de estratégia baseado em turnos, simula o combate entre duas equipes em um pequeno mapa 5x5.
Os personagens se movem aleatoriamente, atacam o inimigo mais próximo e são eliminados ao ter o HP zerado.
O jogo continua até que uma das equipes seja completamente eliminada

Componentes Principais

O projeto é feito em arquivos .h para cabeçalhos e .c para implementações, cada um lida com uma parte específica do jogo.
Onde o main.c é onde rodamos, os personagens, definem a manipulação de personagens, mapas, definem o gerenciamento do tabuleiro e turno é a implementação da fila de turnos.



Mapa.h

Impede inclusões múltiplas para evitar erros de compilação, inclui o “personagem.h”, reconhecendo sua estrutura, define o tamanho do tabuleiro em 5x5.
Respectivamente, as funções preparam o mapa, posicionam os personagens no mapa e imprime o mapa no console.

mapa.h

```h
#ifndef MAPA_H
#define MAPA_H

#include "personagem.h"

#define TAM 5

extern char mapa[TAM][TAM];

void inicializarMapa();
void atualizarMapa(Personagem* raiz_avl);
void exibirMapa();

#endif
```

Mapa.c

A função inicializarMapa() é responsável por preencher o tabuleiro com “.”, represetando os territórios a serem percorridos no tabuleiro.
Em seguida a função atualizarMapa(Personagem* raiz_avl) é responsável por redesenhar o mapa com a posição dos personagens, chamando uma função presente em personagem.c, que vamos tratar mais tardar.
E por fim a função exibirMapa() imprime o conteúdo do mapa no console.

mapa.c 2

mapa.c > exibirMapa()

```c
#include <stdio.h>
#include "mapa.h"
#include "personagem.h"

char mapa[TAM][TAM];

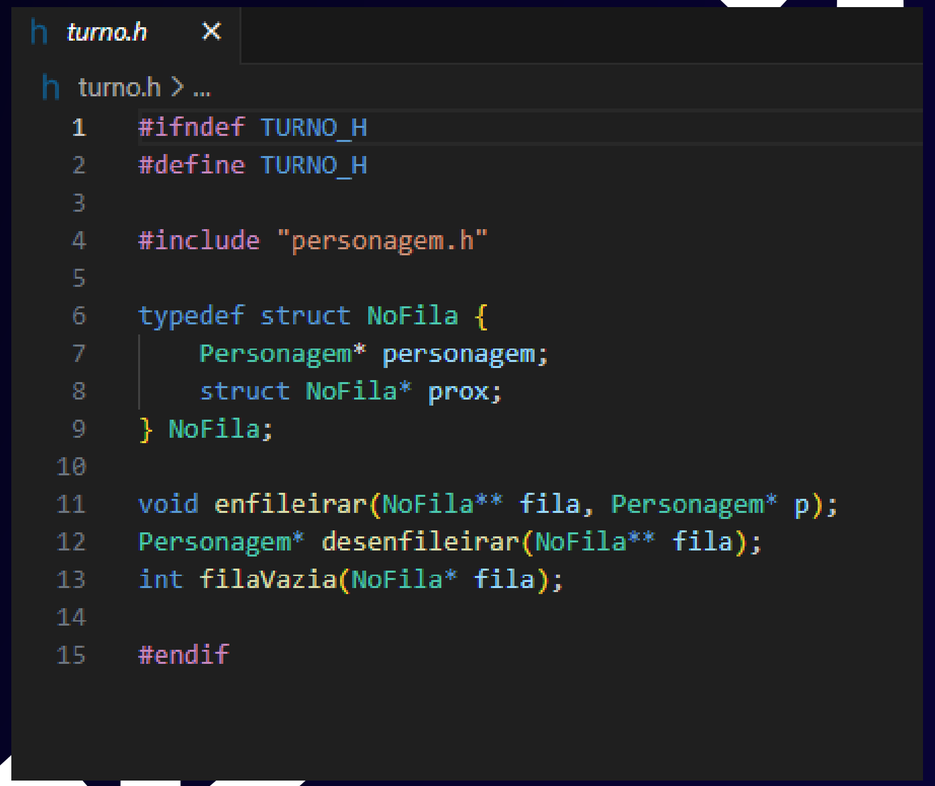
void inicializarMapa() {
    for (int i = 0; i < TAM; i++)
        for (int j = 0; j < TAM; j++)
            mapa[i][j] = '.';
}


void atualizarMapa(Personagem* raiz_avl) {
    inicializarMapa();
    atualizarMapaAVL(raiz_avl);
}

void exibirMapa() {
    printf("\nMapa:\n");
    for (int i = 0; i < TAM; i++) {
        for (int j = 0; j < TAM; j++) {
            printf("%c ", mapa[i][j]);
        }
        printf("\n");
    }
}
```

Turno.h

Guarda a inclusão, essencial para evitar múltiplas inclusões, inclui personagem.h, necessário para que os nós da fila possam referenciar objetos Personagem.
Sobre a struct NoFila, contém um ponteiro para o personagem associado a este turno, e contém um outro ponteiro para o próximo nó na sequência.
Chama funções nas quais tratarei mais tardar.



Turno.c

Aqui implementa as operações dos turnos:
A função enfileirar cria um novo nó e armazena em Personagem* p, se a fila estivar vazia, o novo nó se torna a cabeça, caso contrário, percorre a fila até o final e anexa o novo nó lá.
A função desinfileirar remove o primeiro nó da fila e retorna o Personagem* associado, atualiza a cabeça da fila para o próximo nó, libera a memória do nó removido e retorna NULL se a fila estiver vaiza.
Por fim, a função filaVazia verifica de forma simples se a fila está vazia.

turno.c 2

turno.c > ...

```c
#include <stdlib.h>
#include "turno.h"

void enfileirar(NoFila** fila, Personagem* p) {
    NoFila* novo = malloc(sizeof(NoFila));
    novo->personagem = p;
    novo->prox = NULL;

    if (*fila == NULL) {
        *fila = novo;
    } else {
        NoFila* temp = *fila;
        while (temp->prox)
            temp = temp->prox;
        temp->prox = novo;
    }
}

Personagem* desenfileirar(NoFila** fila) {
    if (*fila == NULL) return NULL;
    NoFila* temp = *fila;
    Personagem* p = temp->personagem;
    *fila = temp->prox;
    free(temp);
    return p;
}

int filaVazia(NoFila* fila) {
    return fila == NULL;
}
```

Personagem.h

Aqui, definimos a estrutura fundamental dos personagens.
Atribui os atributos de jogo, os ponteiros necessários e a altura do nó para o balanceamento AVL.
Aloca e inicializa um novo personagem, simula ataques, altera a posição do personagem, adiciona um personagem à árvore, mantendo o balanceamento, remove um personagem da árvore, libera toda a memória, remove personagens mortos da árvore.
GetAltura e getBalanceamento são função auxiliares para o balanceamento da AVL.
Implementa rotações para manter o balanceamento, busca o alvo mais próximo, conta os personagens vivos e percore a árvore para atualizar as posições no mapa.

personagem.h

personagem.h > removerMortosAVL(Personagem *)

```h
#ifndef PERSONAGEM_H
#define PERSONAGEM_H

typedef struct Personagem {
    char nome[30];
    int hp;
    int ataque;
    int x, y;
    int equipe;

    struct Personagem* left;
    struct Personagem* right;
    int height;
} Personagem;

Personagem* criarPersonagem(const char* nome, int hp, int ataque, int x, int y, int equipe);
void atacar(Personagem* atacante, Personagem* alvo);
void mover(Personagem* p, int dx, int dy);

Personagem* inserirPersonagemAVL(Personagem* no, Personagem* novoPersonagem);

Personagem* removerPersonagemAVL(Personagem* no, const char* nome);

void liberarArvoreAVL(Personagem* no);

Personagem* removerMortosAVL(Personagem* raiz);

int getAltura(Personagem* no);

int getBalanceamento(Personagem* no);

Personagem* rotacaoDireita(Personagem* y);
Personagem* rotacaoEsquerda(Personagem* x);

Personagem* encontrarAlvoProximoAVL(Personagem* atacante, Personagem* raizArvore);

void contarEquipesAVL(Personagem* no, int* herois, int* inimigos);

void atualizarMapaAVL(Personagem* no);

#endif
```

Personagem.c (parte 1)

Este é o maior código do projeto, por isso vai ser dividido em algumas partes. Possui as todas as funcionalidades de personagens e a implementação da árvore AVL.
Aqui a principal função, onde aloca e inicializa um personagem, configurando-o como um nó AVL.
Em seguida possui funções como getAltura, onde retorna a altura, atualizarAltura, onde atualiza a altura, getBalanceamento, onde calcula o fator de balanceamento, encontrarMinimoAVL, onde percorre a subárvore esquerda para encontrar o nó com a menor chave.

```c
Personagem* criarPersonagem(const char* nome, int hp, int ataque, int x, int y, int equipe) {
    Personagem* p = (Personagem*) malloc(sizeof(Personagem));
    if (p == NULL) {
        fprintf(stderr, "Erro: Falha na alocação de memória para Personagem '%s'.\n", nome);
        exit(EXIT_FAILURE);
    }
    strcpy(p->nome, nome);
    p->hp = hp;
    p->ataque = ataque;
    p->x = x;
    p->y = y;
    p->equipe = equipe;
    p->left = NULL;
    p->right = NULL;
    p->height = 1;
    return p;
}
```

Personagem.c (Parte 2)

Funções de rotações, que implementam rotações simples, movimentos essenciais para reequilibrar a árvore AVL. São chamadas quando o fator de balanceamento excede os limites permitidos.

```c
Personagem* rotacaoDireita(Personagem* y) {
    Personagem* x = y->left;
    Personagem* T2 = x->right;

    x->right = y;
    y->left = T2;

    atualizarAltura(y);
    atualizarAltura(x);

    return x;
}

Personagem* rotacaoEsquerda(Personagem* x) {
    Personagem* y = x->right;
    Personagem* T2 = y->left;

    y->left = x;
    x->right = T2;

    atualizarAltura(x);
    atualizarAltura(y);

    return y;
}
```

Personagem.c (Parte 3)

Aqui é inserido um novo personagem na árvore de forma recursiva, mantendo a ordem alfabética dos nomes.
Após a inserção, verifica e executa as rotações necessárias para manter a árvore balanceada.
Em seguida possui a função removerPersonagem, onde remove e rebalanceia a árvore com rotações se necessário.
Depois dessas funções, vem funções envolvendo a árvore AVL, como liberarArvoreAVL, que libera a memória, removerMortosAvl, que remove um personagem com hp <= 0 da árvore.

```c
Personagem* inserirPersonagemAVL(Personagem* no, Personagem* novoPersonagem) {
    if (no == NULL)
        return novoPersonagem;

    int cmp = strcmp(novoPersonagem->nome, no->nome);

    if (cmp < 0)
        no->left = inserirPersonagemAVL(no->left, novoPersonagem);
    else if (cmp > 0)
        no->right = inserirPersonagemAVL(no->right, novoPersonagem);
    else
        return no;
    atualizarAltura(no);

    int balanceamento = getBalanceamento(no);

    if (balanceamento > 1 && strcmp(novoPersonagem->nome, no->left->nome) < 0)
        return rotacaoDireita(no);

    if (balanceamento < -1 && strcmp(novoPersonagem->nome, no->right->nome) > 0)
        return rotacaoEsquerda(no);

    if (balanceamento > 1 && strcmp(novoPersonagem->nome, no->left->nome) > 0) {
        no->left = rotacaoEsquerda(no->left);
        return rotacaoDireita(no);
    }

    if (balanceamento < -1 && strcmp(novoPersonagem->nome, no->right->nome) < 0) {
        no->right = rotacaoDireita(no->right);
        return rotacaoEsquerda(no);
    }

    return no;
}
```

Main.c (Parte 1)

A função checarFim determina se o jogo acabou. Utiliza contarEquipesAVL para verificar se alguma das equipes não possui mais personagens vivos na AVL.
Após isso, inicializa um gerador de números aleatórios para movimentos e ações imprevisíveis dos personagens, gerencia os personagens ativos e organiza sequência de ações dos personagens. Chama a função inicializarMapa, onde limpa o grid preenchendo com “.” . Cria 2 personagens heróis e 2 inimigos, onde são inseridos na AVL. Em seguida, os personagens são adicionados à fila de turnos usando a função enfileirar, definindo a ordem em que agirão inicialmente, junto com um contador de turnos, que será exibido no final.

main.c > main()

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "personagem.h"
#include "mapa.h"
#include "turno.h"

int checarFim(Personagem* raiz_avl) {
    int herois = 0, inimigos = 0;
    contarEquipesAVL(raiz_avl, &herois, &inimigos);
    return (herois == 0 || inimigos == 0);
}

int main() {
    srand(time(NULL));

    Personagem* raiz_avl = NULL;
    NoFila* fila = NULL;

    inicializarMapa();

    Personagem* h1 = criarPersonagem("Heroi1", 10, 3, 0, 0, 0);
    Personagem* h2 = criarPersonagem("Heroi2", 10, 2, 0, 1, 0);
    Personagem* i1 = criarPersonagem("Inimigo1", 8, 2, 4, 4, 1);
    Personagem* i2 = criarPersonagem("Inimigo2", 9, 3, 4, 3, 1);

    raiz_avl = inserirPersonagemAVL(raiz_avl, h1);
    raiz_avl = inserirPersonagemAVL(raiz_avl, h2);
    raiz_avl = inserirPersonagemAVL(raiz_avl, i1);
    raiz_avl = inserirPersonagemAVL(raiz_avl, i2);

    enfileirar(&fila, h1);
    enfileirar(&fila, h2);
    enfileirar(&fila, i1);
    enfileirar(&fila, i2);

    int turno_num = 1;
```

Main.c (Parte 2)

Aqui possuí o principal loop do jogo, que continua até que uma das equipes for completamente eliminada. Inicia o turno mostrando o número atual, redesenha o mapa e mostra o estado dos personagens.
O próximo personagem a agir é retirado da fila de turnos, o turno é processado apenas se o personagem atual estiver vivo, caso contrário ele é removido da fila. Possui a ação de ataque, que tenta localizar um inimigo vivo próximo, se for encontrado a função atacar é chamada. Se nenhum alvo for encontrado o personagem se move aleatoriamente.
No pós-turno, varre a lista de personagens e se tiver algum morto, é removido permanentemente.
O loop termina quando uma equipe é eliminada, libera a memória e encerra o jogo.

```c
    while (!checarFim(raiz_avl)) {
        printf("\n--- TURNO %d ---\n", turno_num++);
        atualizarMapa(raiz_avl);
        exibirMapa();
        Personagem* atual = desenfileirar(&fila);
        if (atual->hp > 0) {
            printf("\nTurno de %s (HP: %d - Pos: (%d,%d)): ", atual->nome, atual->hp, atual->x, atual->y);
            Personagem* alvo = encontrarAlvoProximoAVL(atual, raiz_avl);
            if (alvo != NULL) {
                atacar(atual, alvo);
            } else {
                int dx = (rand() % 3) - 1;
                int dy = (rand() % 3) - 1;

                int nova_x = atual->x + dx;
                int nova_y = atual->y + dy;

                if (nova_x >= 0 && nova_x < TAM && nova_y >= 0 && nova_y < TAM) {
                    mover(atual, dx, dy);
                    printf("%s moveu para (%d, %d).\n", atual->nome, atual->x, atual->y);
                } else {
                    printf("%s nao pode se mover para fora do mapa.\n", atual->nome);
                }
            }
        } else {
            printf("%s foi removido da fila (ja estava morto).\n", atual->nome);
        }
        raiz_avl = removerMortosAVL(raiz_avl);
        if (atual->hp > 0) {
            enfileirar(&fila, atual);
        } else {
        }
    }
    printf("\nFim de jogo!\n");
    int herois_restantes = 0, inimigos_restantes = 0;
    contarEquipesAVL(raiz_avl, &herois_restantes, &inimigos_restantes);
    if(herois_restantes > 0) {
        printf("Vitoria dos Herois!\n");
    } else if (inimigos_restantes > 0) {
        printf("Vitoria dos Inimigos!\n");
    } else {
        printf("Empate (todos foram eliminados).\n");
    }
    liberarArvoreAVL(raiz_avl);

    return 0;
}
```

Tarefa	Início-Fim	Estimativa de tempo	Observações
Personagem AVL	03/07 - 07/07	7 horas	Foi algo alterado em todo o projeto, por isso demorou tanto tempo.
Implementação de Turnos	03/07 - 04/07	2 horas	Desenvolvimento de turno.c e turno.h
Mapa	04/07 - 04/07	4 horas	Criação das funções do mapa
Ataque/Movimento	03/07 - 05/07	3 horas	Função de ataque e movimento do personagem
Movimentos aleatórios	06/07 - 06/07	1 hora	Ideia para movimentos aleatórios
Construção do loop principal	06/07 - 08/07	4 horas	Implementação do ciclo principal do jogo
Finalização/Testes	07/07 - 08/07	4 horas	Tive alguns problemas ao rodar, porém consegui solucionar os problemas e completar o código

Obrigado